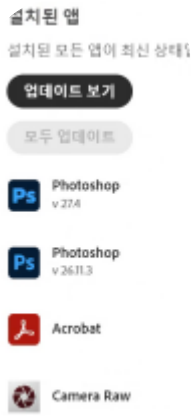
이미지에 있는 문구 **"오신 것을 환영합니다."**는 한국어로,

👉 **"Welcome" / "You are welcome (we welcome you)"** 라는 뜻입니다.



이미지에 보이는 내용은 설치 프로그램의 메뉴로 보이고, 한국어를 해석하면 다음과 같습니다:

- **설치됨 앱** → Installed Apps (이미 설치된 앱)
- **설치된 모든 앱의 최신 상태를 확인하세요** → Check the latest status of all installed apps
- **업데이트 보기** → View Updates
- **모두 업데이트** → Update All

아래에는 설치된 프로그램 목록이 보입니다:

- Photoshop (버전 27.4)
- Photoshop (버전 26.11.3)
- Acrobat
- Camera Raw

즉, 현재 설치된 Adobe 앱들과 업데이트 상태를 보여주는 화면입니다.

열기

이미지의 버튼에 있는 **"열기"**는 한국어로

👉 **"Open" (열기)** 라는 뜻입니다.

이미지에 있는 한국어 내용을 번역하면 다음과 같습니다:

- **Acrobat 열기** → Open Acrobat
- **업데이트된 새 파일 이름을 선택하면 앱(설치된 경우)이 열립니다.**
  → Select the updated new file name to open the app (if installed).
- **다시 표시 안 함** → Don't show again

버튼:

- **계속** → Continue
- **취소** → Cancel

즉, Acrobat을 열기 전에 나오는 안내 팝업입니다.

What is Derek Software?

"Derek Software" isn't a single well-known universal product—it usually refers to a **specific software product called "The Derek"** (or similar variants) used in the **oil & gas industry**.

Most common meaning

The Derek is:

- A **specialized oil & gas software system** developed by Roughneck Systems SaaS Counter
- Used for **accounting, production tracking, and operational analysis** in energy companies Ruff +1

What it does

Depending on the version, it includes:

- Accounting tools (general ledger, accounts payable/receivable) Roughneck Soft...
- Reporting (financial statements, transaction logs) Roughneck Soft...
- Oil & gas–specific features like:
  - Well and production tracking
  - Royalty and ownership calculations
  - Data analysis for drilling and operations Software World

In simple terms

👉 It's **industry-specific business software** used by oil & gas operators to manage finances, wells, and production data.

Important note

There are a few similarly named things:

- **"Derek Suite" / Dereknet** → a modern data/analytics platform for oil & gas Dereknet AI
- Research tools (like "DEREK" AI modules) → unrelated technical projects

---

✅ **Bottom line:**
When people say "Derek Software," they usually mean **"The Derek" — a legacy but widely used oil & gas accounting and operations software.**

If you tell me where you saw "Derek Software" (e.g., Adobe installer, company tool, etc.), I can narrow it down exactly—because the name can vary by context.

Sources

https://www.lhasalimited.org/solutions/

From the link you shared (Lhasa Limited), "**Derek Software**" specifically refers to:

👉 Derek Nexus (by Lhasa Limited)

What it is

- A **scientific software tool** used to **predict the toxicity (safety risks)** of chemicals. Lhasa Limited
- It's an **"in silico" (computer-based) expert system**, meaning it analyzes chemical structures digitally instead of using lab testing. Lhasa Limited

---

What it actually does

- You input a **chemical compound (e.g., ingredient, drug, impurity)**
- The software checks it against a large **knowledge base of known toxic patterns**
- Then it predicts whether the compound could be harmful

For example, it can predict:

- Cancer risk (carcinogenicity)
- DNA damage (mutagenicity)
- Skin sensitization
- Organ toxicity, etc. Lhasa Limited +1

How it works (simple explanation)

- It looks for "**structural alerts**" → patterns in molecules linked to toxicity
- Compares your compound to known toxic structures
- Gives a **risk prediction + scientific reasoning** Lhasa Limited

---

Who uses it

- Pharmaceutical companies
- Chemical companies
- Cosmetics & personal care (very relevant to ELC Beauty)
- Regulatory scientists Lhasa Limited

Why companies use it

- Reduces need for **animal testing**
- Speeds up **safety assessment**
- Helps with **regulatory compliance (e.g., safety submissions)** Lhasa Limited +1

🧠 In simple terms

👉 **Derek Software = a tool that predicts if a chemical ingredient might be toxic before you test it in real life.**

If you want, I can explain how it's used specifically in **cosmetics safety (which is likely your context)**—that's where tools like Derek are heavily used.

Sources

From where and how to install it

Here's the key thing upfront:
👉 **You cannot directly download "Derek Nexus" like normal software (e.g., from a public download page).**

It is **licensed scientific software** from Lhasa Limited, so installation works differently.

🧭 Where to get it

You must go through the official source:

👉 **Lhasa Limited (official site)**

- Go to their solutions page (you already shared it)
- Then click **"Get in touch" / Contact / Request demo**

They do NOT provide a public download link.

🛠️ How to install Derek Nexus (actual process)

1. Request access / license

- Fill out their contact or demo form
- Or contact Lhasa directly (sales/support)
- They may offer:
  - Trial (e.g., ~2 weeks) Lhasa Limited
  - Full licensed version (paid / enterprise)

---

2. Get credentials + installer

After approval, they will provide:

- Download link (or secure portal access)
- License key / user account
- Installation guide (PDF or support instructions)

---

3. Install on your system

Typical setup (varies by org):

- Run installer (Windows/Linux desktop or server)
- Configure:
  - License server or local license
  - User login
- Optional: integrate with tools like **StarDrop or other Lhasa tools** Optibrium

---

4. Activate & use

- Log in with provided credentials
- Load chemical structures and start predictions

⚠️ Important notes

- ❌ No free public download (not like Adobe, etc.)
- ❌ Not open-source
- ✅ Requires **organization license or membership**
- ✅ Often installed by IT in companies (pharma/cosmetics)

🧠 Simple explanation

👉 Think of Derek Nexus like **enterprise lab software**
—not something you casually install from the internet.

💡 For your situation (important)

Since you're likely in **cosmetics / ELC Beauty context**:

- Your company may **already have a license**
- You might need:
  - Internal access request
  - VPN / license server access
  - IT installation support

✔️ **Best next step:**
Ask internally:

> "Do we have Lhasa Derek Nexus access? How do I get it installed?"

If you want, tell me your setup (company laptop, VM, etc.), and I can walk you through **exact install steps once you get access**.

Sources

What kind of questions will be asked in product based companies for a Software engineer role

Product-based companies (like Google, Amazon, Microsoft, etc.) tend to focus less on rote knowledge and more on **problem-solving, system thinking, and real-world coding ability**.

Here's a clear breakdown of what you can expect 👇

🧠 1. Data Structures & Algorithms (MOST IMPORTANT)

This is the core of most interviews.

Typical questions:

- Arrays & Strings
  → "Find longest substring without repeating characters"
- Linked Lists
  → "Reverse a linked list"
- Trees & Graphs
  → "Lowest common ancestor", "Detect cycle in graph"
- Dynamic Programming
  → "Climbing stairs", "Knapsack"
- Searching & Sorting
  → Binary search variations

👉 Expect:

- 2–3 coding problems per round
- Medium to Hard difficulty (LeetCode style)

💻 2. Coding / Problem Solving Round

You'll be asked to:

- Write clean, working code (no syntax errors ideally)
- Optimize (time & space complexity)
- Explain your approach clearly

👉 Example:

> "Given an array, find two numbers that sum to target."

Then follow-ups:

- Can you do it in O(n)?
- What about duplicates?

🏗️ 3. System Design (for 2+ years experience)

Focus: building scalable systems

Typical questions:

- "Design a URL shortener (like bit.ly)"
- "Design Instagram feed"

- "Design a chat system"

👉 You'll need to discuss:

- APIs
- Databases (SQL vs NoSQL)
- Scalability
- Caching (Redis)
- Load balancing

📚 4. Core Computer Science

Basics matter, especially for freshers.

Topics:

- **Operating Systems**
  - Threads vs processes
  - Deadlocks
- **DBMS**
  - Normalization
  - Indexing
  - Joins
- **Computer Networks**
  - HTTP vs HTTPS
  - TCP/IP
- **OOP**
  - Inheritance, Polymorphism

🧪 5. Coding + Debugging (Practical)

Some companies give:

- Debug this code
- Fix a broken function
- Write test cases

👔 6. Behavioral / HR Round

They check how you think and work.

Common questions:

- "Tell me about yourself"
- "Describe a challenge you faced"
- "Why this company?"
- "Tell me about a failure"

👉 Use **STAR method** (Situation, Task, Action, Result)

🧩 7. Projects Discussion

Very important for freshers.

They may ask:

- "Explain your project"
- "Why did you choose this tech?"
- "What problems did you face?"
- "How would you scale it?"

🔥 What product companies REALLY test

They care about:

✔ Problem-solving ability
✔ Clean coding
✔ Thinking out loud
✔ Trade-offs (not just answers)
✔ Depth over memorization

🚀 Preparation Strategy (simple but effective)

- Practice **DSA daily (LeetCode / CodeStudio)**
- Focus on:
  - Arrays, Strings, Trees, Graphs, DP
- Do **mock interviews**
- Revise **CS fundamentals**
- Be strong in **one programming language**

🧠 Simple summary

👉 Product companies test:

- Can you **solve problems efficiently?**
- Can you **build scalable systems?**
- Can you **think clearly under pressure?**

If you want, I can:

- Give you a **company-wise question list (Amazon, Google, etc.)**
- Or create a **30-day preparation plan tailored to you**

Give you a company-wise question list (Amazon, Google, etc.)

Or create a 30-day preparation plan tailored to you

both one by one

Great—this is exactly the right approach. Let's do it step by step.

🔥 PART 1: Company-wise Question List

I'll break this down into **Amazon, Google, Microsoft, Meta** (most common product companies).

🟠 AMAZON

🔑 Focus Areas

- DSA (medium level mostly)
- **Behavioral (VERY IMPORTANT – Leadership Principles)**

💻 Coding Questions (frequently asked)

- Two Sum / Variants

- Longest Substring Without Repeating Characters
- Merge Intervals
- K Closest Points to Origin
- Top K Frequent Elements
- Number of Islands (Graph)
- LRU Cache
- Sliding Window Maximum

🧠 Behavioral Questions

- "Tell me about a time you handled conflict"
- "When did you take ownership?"
- "Describe a failure"

👉 Must align answers with **Amazon Leadership Principles**

🔵 GOOGLE

🔑 Focus Areas

- Strong DSA (medium → hard)
- Problem-solving depth
- Edge cases

💻 Coding Questions

- Longest Increasing Subsequence
- Word Ladder (Graph BFS)
- Trapping Rain Water
- Median of Two Sorted Arrays
- Serialize/Deserialize Binary Tree
- Course Schedule (Cycle detection)

🧠 They test:

- Thinking process > final answer
- Optimization deeply

👉 Expect follow-ups like:

> "Can you make it O(1) space?"

🟣 MICROSOFT

🔑 Focus Areas

- DSA (medium)
- System design (basic for freshers)
- Clean code

💻 Coding Questions

- Reverse Linked List
- Detect Cycle in Linked List
- Binary Tree Level Order Traversal
- Valid Parentheses
- Merge Two Sorted Lists
- Design HashMap

🧠 Behavioral

- "Explain your project deeply"
- "Why Microsoft?"

👉 They care about **clarity + communication**

🔵 META (Facebook)

🔑 Focus Areas

- DSA (fast + optimized)
- System design (important)

💻 Coding Questions

- Clone Graph
- Binary Tree Right Side View
- Minimum Window Substring
- Add Two Numbers (Linked List)
- Flatten Binary Tree
- Design Facebook News Feed

⚡ Key thing:

👉 Speed matters (you must code fast and correctly)

🟡 COMMON ACROSS ALL

These problems are MUST DO:

- Two Sum
- Binary Search
- DFS / BFS
- Sliding Window
- Recursion + Backtracking
- DP basics

🧠 Reality Check

👉 If you can solve **LeetCode Medium consistently**, you're already in a strong position.

🚀 PART 2: 30-Day Preparation Plan (Tailored)

Assuming:

- You're preparing seriously
- ~3–5 hours/day
- Beginner → Intermediate level

🗓️ WEEK 1: Foundations (VERY IMPORTANT)

Day 1–2

- Arrays basics
- Problems:
  - Two Sum
  - Best Time to Buy/Sell Stock

Day 3–4

- Strings
- Problems:
  - Longest Substring Without Repeating Characters
  - Valid Anagram

Day 5–6

- Linked List
- Problems:
  - Reverse Linked List

- Detect Cycle

Day 7

- Revision + Mock test

🗓️ WEEK 2: Core DSA

Day 8–9

- Stack & Queue
- Problems:
  - Valid Parentheses
  - Min Stack

Day 10–11

- Binary Search
- Problems:
  - Search in Rotated Array

Day 12–14

- Trees (VERY IMPORTANT)
- Problems:
  - Inorder Traversal
  - Lowest Common Ancestor
  - Level Order Traversal

🗓️ WEEK 3: Advanced Topics

Day 15–16

- Graphs
- Problems:
  - Number of Islands
  - BFS/DFS

Day 17–18

- Sliding Window
- Problems:
  - Maximum Subarray
  - Longest Subarray

Day 19–21

- Dynamic Programming (start simple)
- Problems:
  - Climbing Stairs
  - House Robber

📅 WEEK 4: Interview Level

Day 22–24

- Mixed problems (medium level)
- Timed practice (important)

Day 25–26

- System Design (basic)
- Learn:
  - URL shortener
  - Caching

Day 27–28

- Mock Interviews
- Practice explaining solutions

Day 29

- Behavioral prep
- Prepare stories (STAR method)

Day 30

- Full revision + confidence boost

🧠 Daily Routine (important)

Each day:

1. Learn concept (30–60 min)
2. Solve 3–5 problems
3. Revise old problems
4. Explain out loud

🔥 Final Advice

- Don't just solve → **understand patterns**
- Re-solve problems after 2–3 days
- Focus on **consistency** > **intensity**

If you want next level:
👉 I can give you:

- A **company-specific Amazon prep plan**
- OR a **list of exact LeetCode problems (top 75/150)**
- OR mock interview questions with answers

Just tell me 👍

list of exact LeetCode problems (top 75/150)

Thought for 15s >

Here are the two canonical lists people usually mean:

- **LeetCode 75**: LeetCode's official 75-question study plan, aimed at **1–3 months** of prep. LeetCode
- **Top Interview 150**: LeetCode's official 150-question study plan, aimed at **3+ months** of prep. LeetCode +1

Also, if by "75" you meant **Blind 75**, that exists too as a separate community-curated list on LeetCode. LeetCode +1

1) Official LeetCode 75

ChatGPT

Array / String

1. Merge Strings Alternately
2. Greatest Common Divisor of Strings
3. Kids With the Greatest Number of Candies
4. Can Place Flowers
5. Reverse Vowels of a String

6. Reverse Words in a String
7. Product of Array Except Self
8. Increasing Triplet Subsequence
9. String Compression

Two Pointers

10. Move Zeroes
11. Is Subsequence
12. Container With Most Water

Sliding Window

13. Maximum Average Subarray I
14. Maximum Number of Vowels in a Substring of Given Length
15. Max Consecutive Ones III
16. Longest Subarray of 1's After Deleting One Element

Prefix Sum

17. Find the Highest Altitude
18. Find Pivot Index

Hash Map / Set

19. Find the Difference of Two Arrays
20. Unique Number of Occurrences
21. Determine if Two Strings Are Close
22. Equal Row and Column Pairs

Stack

23. Removing Stars From a String
24. Asteroid Collision
25. Decode String

Queue

26. Number of Recent Calls
27. Dota2 Senate

Linked List

28. Delete the Middle Node of a Linked List
29. Odd Even Linked List
30. Reverse Linked List
31. Maximum Twin Sum of a Linked List

Binary Tree – DFS

32. Maximum Depth of Binary Tree

33. Leaf-Similar Trees
34. Count Good Nodes in Binary Tree
35. Path Sum III
36. Longest ZigZag Path in a Binary Tree
37. Lowest Common Ancestor of a Binary Tree

Binary Tree – BFS

38. Binary Tree Right Side View
39. Maximum Level Sum of a Binary Tree

Binary Search Tree

40. Search in a Binary Search Tree
41. Delete Node in a BST

Graphs – DFS

42. Keys and Rooms
43. Number of Provinces
44. Reorder Routes to Make All Paths Lead to the City Zero
45. Evaluate Division

Graphs – BFS

46. Nearest Exit from Entrance in Maze
47. Rotting Oranges

Heap / Priority Queue

48. Kth Largest Element in an Array
49. Smallest Number in Infinite Set
50. Maximum Subsequence Score
51. Total Cost to Hire K Workers

Binary Search

52. Guess Number Higher or Lower
53. Successful Pairs of Spells and Potions
54. Find Peak Element
55. Koko Eating Bananas

Backtracking

56. Letter Combinations of a Phone Number
57. Combination Sum III

Dynamic Programming – 1D

58. N-th Tribonacci Number
59. Min Cost Climbing Stairs

60. House Robber
61. Domino and Tromino Tiling

Dynamic Programming – Multidimensional
62. Unique Paths
63. Longest Common Subsequence
64. Best Time to Buy and Sell Stock with Transaction Fee
65. Edit Distance

Bit Manipulation
66. Counting Bits
67. Single Number
68. Minimum Flips to Make a OR b Equal to c

Trie
69. Implement Trie (Prefix Tree)
70. Search Suggestions System

Intervals
71. Non-overlapping Intervals
72. Minimum Number of Arrows to Burst Balloons

Monotonic Stack
73. Daily Temperatures
74. Online Stock Span

Math
75. Maximum Number of Achievable Transfer Requests
(Depending on the current LeetCode page version, the last few category labels/order can vary slightly, but the official plan remains the source of truth.) LeetCode

2) Official Top Interview 150

Source: LeetCode Top Interview 150 study plan / problemset. LeetCode +1

I'm listing these in the usual grouped order used in the LeetCode plan.

Array / String

1. Merge Sorted Array
2. Remove Element
3. Remove Duplicates from Sorted Array
4. Remove Duplicates from Sorted Array II
5. Majority Element

6. Rotate Array
7. Best Time to Buy and Sell Stock
8. Best Time to Buy and Sell Stock II
9. Jump Game
10. Jump Game II
11. H-Index
12. Insert Delete GetRandom O(1)
13. Product of Array Except Self
14. Gas Station
15. Candy
16. Trapping Rain Water
17. Roman to Integer
18. Integer to Roman
19. Length of Last Word
20. Longest Common Prefix
21. Reverse Words in a String
22. Zigzag Conversion
23. Find the Index of the First Occurrence in a String
24. Text Justification

Two Pointers

25. Valid Palindrome
26. Is Subsequence
27. Two Sum II - Input Array Is Sorted
28. Container With Most Water
29. 3Sum

Sliding Window

30. Minimum Size Subarray Sum
31. Longest Substring Without Repeating Characters
32. Substring with Concatenation of All Words
33. Minimum Window Substring

Matrix

34. Valid Sudoku
35. Spiral Matrix
36. Rotate Image
37. Set Matrix Zeroes
38. Game of Life

Hashmap

39. Ransom Note

40. Isomorphic Strings
41. Word Pattern
42. Valid Anagram
43. Group Anagrams
44. Two Sum
45. Happy Number
46. Contains Duplicate II
47. Longest Consecutive Sequence

Intervals

48. Summary Ranges
49. Merge Intervals
50. Insert Interval
51. Minimum Number of Arrows to Burst Balloons

Stack

52. Valid Parentheses
53. Simplify Path
54. Min Stack
55. Evaluate Reverse Polish Notation
56. Basic Calculator

Linked List

57. Linked List Cycle
58. Add Two Numbers
59. Merge Two Sorted Lists
60. Copy List with Random Pointer
61. Reverse Linked List II
62. Reverse Nodes in k-Group
63. Remove Nth Node From End of List
64. Delete Duplicates from Sorted List II
65. Rotate List
66. Partition List
67. LRU Cache

Binary Tree – General

68. Maximum Depth of Binary Tree
69. Same Tree
70. Invert Binary Tree
71. Symmetric Tree
72. Construct Binary Tree from Preorder and Inorder Traversal

73. Construct Binary Tree from Inorder and Postorder Traversal
74. Populating Next Right Pointers in Each Node II
75. Flatten Binary Tree to Linked List
76. Path Sum
77. Sum Root to Leaf Numbers
78. Binary Tree Maximum Path Sum
79. Binary Search Tree Iterator
80. Count Complete Tree Nodes
81. Lowest Common Ancestor of a Binary Tree

Binary Tree – BFS

82. Binary Tree Level Order Traversal
83. Binary Tree Zigzag Level Order Traversal
84. Binary Tree Right Side View
85. Average of Levels in Binary Tree
86. Binary Tree Level Order Traversal II

Binary Search Tree

87. Minimum Absolute Difference in BST
88. Kth Smallest Element in a BST
89. Validate Binary Search Tree

Graph – General

90. Number of Islands
91. Surrounded Regions
92. Clone Graph
93. Evaluate Division
94. Course Schedule
95. Course Schedule II
96. Snakes and Ladders
97. Minimum Genetic Mutation
98. Word Ladder

Trie

99. Implement Trie (Prefix Tree)
100. Design Add and Search Words Data Structure
101. Word Search II

Backtracking

102. Letter Combinations of a Phone Number
103. Combinations
104. Permutations

105. Combination Sum
106. N-Queens
107. Generate Parentheses
108. Word Search

Divide and Conquer

109. Convert Sorted Array to Binary Search Tree
110. Sort List
111. Construct Quad Tree
112. Merge k Sorted Lists
113. Maximum Subarray

Kadane's Algorithm

114. Maximum Sum Circular Subarray

Binary Search

115. Search Insert Position
116. Search a 2D Matrix
117. Find Peak Element
118. Search in Rotated Sorted Array
119. Find First and Last Position of Element in Sorted Array
120. Find Minimum in Rotated Sorted Array
121. Median of Two Sorted Arrays

Heap

122. Kth Largest Element in an Array
123. IPO
124. Find K Pairs with Smallest Sums
125. Find Median from Data Stream

Bit Manipulation

126. Add Binary
127. Reverse Bits
128. Number of 1 Bits
129. Single Number
130. Single Number II
131. Bitwise AND of Numbers Range
132. Palindrome Number

Math

133. Plus One
134. Factorial Trailing Zeroes

135. Sqrt(x)
136. Pow(x, n)
137. Max Points on a Line

1D Dynamic Programming

138. Climbing Stairs
139. House Robber
140. Word Break
141. Coin Change
142. Longest Increasing Subsequence

Multidimensional DP

143. Triangle
144. Minimum Path Sum
145. Unique Paths II
146. Longest Palindromic Substring
147. Interleaving String
148. Edit Distance
149. Best Time to Buy and Sell Stock III
150. Best Time to Buy and Sell Stock IV

3) If you meant Blind 75 instead of LeetCode 75

Blind 75 is a separate, very popular curated list; LeetCode hosts a version of it, and the classic list is also widely referenced in LeetCode Discuss. LeetCode +1

The classic Blind 75 includes staples like:

- Two Sum
- Best Time to Buy and Sell Stock
- Contains Duplicate
- Product of Array Except Self
- Maximum Subarray
- Maximum Product Subarray
- Find Minimum in Rotated Sorted Array
- Search in Rotated Sorted Array
- 3Sum
- Container With Most Water
- Climbing Stairs
- Coin Change
- Longest Increasing Subsequence
- Longest Common Subsequence

- Word Break
- Combination Sum
- House Robber / House Robber II
- Decode Ways
- Unique Paths
- Clone Graph
- Course Schedule
- Number of Islands
- Longest Consecutive Sequence
- Reverse Linked List
- Detect Cycle in a Linked List
- Merge Two Sorted Lists
- Remove Nth Node From End of List
- Reorder List
- Set Matrix Zeroes
- Spiral Matrix
- Rotate Image
- Word Search
- Longest Substring Without Repeating Characters
- Longest Repeating Character Replacement
- Minimum Window Substring
- Valid Anagram
- Group Anagrams
- Valid Parentheses
- Valid Palindrome
- Longest Palindromic Substring
- Palindromic Substrings
- Maximum Depth of Binary Tree
- Same Tree
- Invert Binary Tree
- Binary Tree Maximum Path Sum
- Binary Tree Level Order Traversal
- Serialize and Deserialize Binary Tree
- Subtree of Another Tree
- Construct Binary Tree from Preorder and Inorder Traversal
- Validate Binary Search Tree
- Kth Smallest Element in a BST
- Lowest Common Ancestor of BST
- Implement Trie
- Add and Search Word

- Word Search II
- Top K Frequent Elements
- Find Median from Data Stream
  …and the remaining standard Blind 75 set as maintained on LeetCode / community lists. LeetCode +1

What I'd recommend you do

For interviews:

- Start with **LeetCode 75** if you want a manageable, structured base. LeetCode
- Move to **Top Interview 150** once you're comfortable with mediums. LeetCode +1
- Use **Blind 75** as a "must-not-miss" checklist. LeetCode +1

Next, I can turn this into a **day-by-day checklist with problem links and difficulty order**.

Sources